Design Pattern Authenticator

Intento e definizione

L'intento del design pattern Authenticator è verificare che un soggetto (utente o sistema) che tenta di accedere al sistema sia effettivamente chi dichiara di essere. Si applica in contesti dove il sistema custodisce risorse di valore (come, ad esempio, dati aziendali medici, ecc.) e l'accesso deve essere limitato solo a chi ha un motivo legittimo.

Il **problema** principale è prevenire l'accesso da parte di impostori, bilanciando diverse necessità (o "forze"):

1. **Flessibilità**: il sistema deve poter gestire vari tipi di utenti e livelli di sicurezza e riservatezza diversi per diverse risorse.
2. **Affidabilità** (dependability): il protocollo di autenticazione deve essere robusto e sicuro, quindi affidabile.
3. **Costo**: più un sistema è sicuro, più tende ad essere costoso (in termini di risorse o usabilità).
4. **Prestazioni**: se l'autenticazione dovrà essere fatta frequentemente, le prestazioni sono da tenere in considerazione.
5. **Frequenza**: non si può chiedere all'utente di autenticarsi troppo frequentemente.

La **soluzione** al problema proposta dal design pattern Authenticator è utilizzare un **singolo punto di accesso** (*Single Point of Access*) che intercetta le interazioni del soggetto. Questo componente applica un protocollo per verificare l'identità e, se la verifica ha successo, genera una prova di identità (*Proof of Identity*).

Struttura e partecipanti

Il pattern si compone di quattro partecipanti principali:

1. **Subject**: chi richiede l'accesso al sistema (come un utente, un servizio o un sistema esterno) tramite login.
2. **Authenticator**: il componente critico del sistema che riceve la richiesta e applica il protocollo di verifica. Costituisce quindi il Single Point of Access per l'autenticazione che ha la responsabilità di ricevere le credenziali, verificarle e generare una prova di identità (nel caso in cui l'autenticazione è soddisfatta).
3. **AuthenticationInfo**: le informazioni segrete o note (come password hashate, dati biometrici o chiavi pubbliche) usate per validare l'utente e verificare la sua identità. Queste informazioni non dovrebbero mai essere password in chiaro, ma devono essere protette e conservate in modo sicuro.
4. **ProofOfIdentity**: un oggetto (come una sessione, un token JWT o una credenziale temporanea) creato dall'Authenticator se la verifica ha successo. Viene assegnato al soggetto per permettergli di accedere alle risorse senza doversi ri-autenticare continuamente. Serve quindi per evitare di ripetere il login a ogni richiesta.

Funzionamento

Il workflow tipico del pattern Authenticator è il seguente:

1. L'utente invia una richiesta di login() all'Authenticator.
2. L'Authenticator esegue verify() confrontando le credenziali fornite con le AuthenticationInfo.
3. Se i dati corrispondono, l'Authenticator crea (create()) una ProofOfIdentity (es. un token).
4. La prova viene restituita all'utente per le sessioni future.

Diagramma UML di sequenza

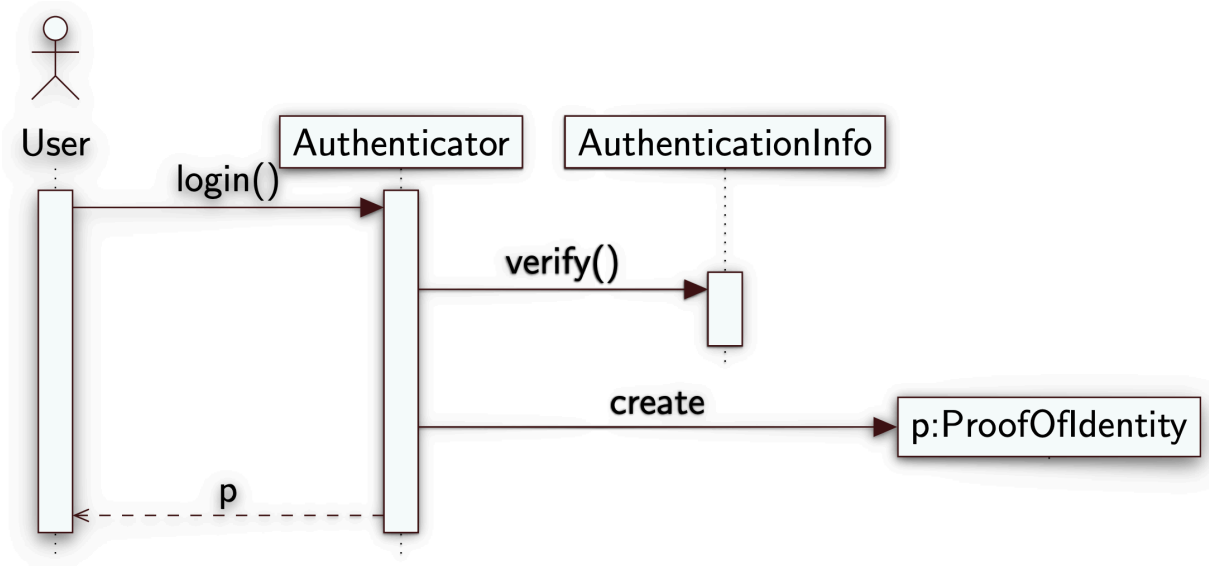
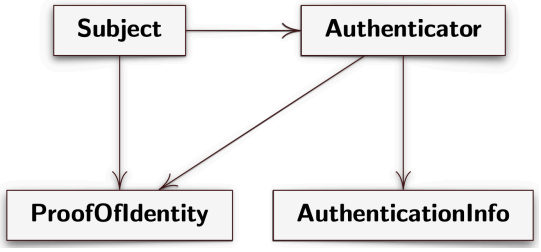


Diagramma UML delle classi



Conseguenze

L'uso del pattern Authenticator porta i seguenti benefici:

- **Flessibilità**: è possibile cambiare il meccanismo di autenticazione (password, smart card, biometria) modificando solo l'Authenticator e le AutenticationInfo, senza toccare il resto del sistema. Cioè si possono gestire tanti tipi di utenti autenticandoli in diversi modi.
- **Dependability**: le informazioni sensibili di autenticazione sono separate dalla logica di business e possono essere memorizzate in aree protette, alle quali tutti hanno accesso in sola lettura.
- **Costo**: possiamo usare una varietà di algoritmi e protocolli per autenticazione con diversa solidità. Le tre varianti esistenti includono ciò che l'utente conosce, ciò che l'utente possiede e ciò che l'utente è.
- **Performance e frequenza**: grazie alla generazione della ProofOfIdentity (token), l'autenticazione pesante viene eseguita una sola volta, così da non dover fare nuovamente l'autenticazione, migliorando le prestazioni per le richieste successive.

Variante dell'Authenticator

Una variante comune dell'Authenticator è il Single Sign On, dove la verifica dell'identità è valida su più domini per un certo intervallo di tempo. Il risultato dell'autenticazione è una credenziale (token identificativo), usata per qualificare i futuri accessi dell'utente nel sistema distribuito.

Hashing di password

Quando un utente si registra ad un servizio, la password fornita deve essere memorizzata per le future autenticazioni. Tuttavia, per motivi di sicurezza, non si può conservare la password in chiaro. Si conserva invece un "derivato" della password, noto come **codice hash**, che funge da AuthenticationInfo.

Una funzione di hashing sicura deve possedere le seguenti caratteristiche:

- **Deterministica**: lo stesso input deve produrre sempre lo stesso output.
- **Irreversibile**: non deve essere possibile risalire al dato originale (la password) partendo dall'hash.
- **Lunghezza fissa**: l'output deve avere una dimensione fissa, indipendentemente dalla lunghezza dell'input.
- **Sensibile alle modifiche**: anche un cambiamento minimo nell'input (ad esempio di un solo carattere) deve generare un hash completamente diverso.

Hashing function e lentezza

Un requisito fondamentale per l'hashing delle password è che la funzione sia **computazionalmente lenta**. Questo serve a contrastare gli attacchi di forza bruta (brute force), in cui un attaccante prova ad applicare la funzione di hashing a un dizionario di password fino a trovare una corrispondenza.

Se la funzione è lenta, verificare un intero dizionario diventa impraticabile.

- *Esempio:* Se la funzione impiega 100 ms per un singolo hash (tempo accettabile per il login di un utente), un attaccante che deve testare un dizionario di 10^9 password impiegherebbe circa 3 anni e 2 mesi (10^8 secondi) per completare l'attacco.

Hashing con salt

L'uso del salt consiste nell'aggiungere un valore casuale alla password prima di calcolarne l'hash. Questo meccanismo risolve due problemi principali:

1. impedisce che due utenti con la stessa password abbiano lo stesso hash memorizzato.
2. difende contro le rainbow tables (database precalcolati e compressi di hash), costringendo l'attaccante a ricalcolare le tabelle per ogni singolo salt (attacco impraticabile).

Naturalmente, per verificare le credenziali in futuro, il sistema deve conoscere il salt che è stato usato per quello specifico utente. Il salt infatti non è un valore segreto (di solito viene salvato insieme all'hash).

Libreria SCrypt

Scrypt è una funzione di hashing progettata specificamente per essere lenta e per consumare risorse di CPU e memoria (*memory-hard*), rendendo costosi gli attacchi hardware dedicati.

Scrypt utilizza tre parametri principali per configurare il suo funzionamento:

- **N** (costo CPU/Memoria): definisce il lavoro da svolgere. L'uso di memoria e CPU cresce linearmente con N. Se N è una potenza di 2, l'operazione è ottimizzata. La memoria utilizzata è pari a $128 \times N \times r$ bytes.
- **r** (block size): influenza la lunghezza del risultato e la memoria utilizzata.
- **p** (parallelizzazione): indica il fattore di parallelizzazione. L'operazione può essere svolta sequenzialmente (richiedendo p volte il tempo) o in parallelo (richiedendo p volte la memoria).

Uso di SCrypt

È possibile utilizzare la classe SCryptUtil per creare e verificare gli hash.

Creazione dell'hash

Per generare l'hash di una password, si invoca il metodo scrypt passando i parametri discussi sopra (N, r, p):

```
final String hash = SCryptUtil.scrypt(passwd, 32768, 8, 1);
```

Il risultato è una stringa che contiene parametri, salt e hash finale (tutto insieme).

Verifica della password

Per verificare se una password fornita durante il login corrisponde all'hash salvato (AuthenticationInfo), si usa il metodo check:

```
SCryptUtil.check(passwd, authenticationInfo);
```

Internamente, il metodo check estrae il salt originale dall'hash memorizzato (authenticationInfo) e lo utilizza per applicare la funzione di hash alla password inserita (passwd), confrontando poi i risultati e restituisce true o false. Questo è molto utile perché ci permette di non dover gestire manualmente il salt e non ricordarci i parametri utilizzati da SCryptUtil.scrypt.

Attacco Denial of Service (DoS)

Un attacco DoS mira ad impedire agli utenti legittimi di utilizzare un servizio. L'attaccante genera solitamente un traffico enorme che sopraffà la capacità del server, rendendolo irraggiungibile.

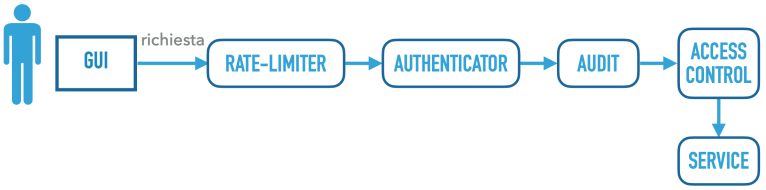
Esistono varianti più complesse di un attacco DoS, come:

- **DDoS** (Distributed DoS): l'attacco proviene da molte macchine diverse sparse su Internet (spesso botnet), rendendo molto più difficile bloccare il traffico malevolo rispetto ad un singolo client.
- Amplificazione (es. **DNS Amplification**): il client nocivo invia piccole richieste a servizi terzi (come i server DNS) falsificando l'indirizzo di ritorno (spoofing) con quello della vittima. La vittima viene così inondata dalle risposte (molto più grandi delle richieste) senza aver mai inviato nulla.

Rate-Limiting

Per difendersi da questi attacchi e garantire che il sistema abbia risorse sufficienti, si utilizza il rate-limiting. Questa tecnica limita la frequenza delle richieste consentite. Se non si è sicuri di avere risorse sufficienti, la richiesta non deve arrivare al servizio.

- **Posizionamento**: poiché anche la verifica dell'identità consuma risorse, il rate-limiting va applicato prima di qualsiasi altra elaborazione (autenticazione, audit, ecc.).



- **Architettura**: idealmente, il componente che limita le richieste si trova a livello di **gateway** o **reverse proxy** (così il traffico viene bloccato prima che entri nel sistema interno lato server). Nel diagramma UML il componente è indicato come "ServerGateway" (punto d'accesso lato client).
- **Defence in Depth**: per una sicurezza maggiore, si può aggiungere una limitazione anche sui singoli server interni, senza fidarci così di un singolo livello di protezione. In questo modo, se il gateway è mal configurato o compromesso, il sistema interno non crolla (questo è il principio di "*difesa in profondità*").

Guava

La libreria Guava di Google viene utilizzata per implementare facilmente il rate-limiting in Java. La classe principale è RateLimiter.

Creazione

Si definisce il numero di richieste per secondo (sotto sono 2) consentite:

```
RateLimiter rateLimiter = RateLimiter.create(2);
```

Utilizzo bloccante

Il metodo `acquire()` blocca il richiedente che supera il limite, mettendolo in attesa finché non ottiene nuovamente il "permesso" di procedere:

```
rateLimiter.acquire();
```

Utilizzo con timeout

Si può provare ad acquisire il permesso con un tempo massimo di attesa; se scade, la richiesta viene respinta (utile per non bloccare thread all'infinito):

```
rateLimiter.tryAcquire(10, TimeUnit.MICROSECONDS);
```

Riepilogo del flusso (lato server)

Per ogni richiesta in arrivo, il server deve seguire rigorosamente questo ordine di operazioni per garantire la sicurezza:

1. **Rate-Limiting**: limitare il flusso di richieste in ingresso.
2. **Autenticazione**: verificare l'identità dell'utente (tramite credenziali o token).
3. **Autorizzazione**: verificare se l'utente ha i permessi per l'operazione richiesta.

4. **Esecuzione**: solo alla fine, processare la richiesta vera e propria ("Service").

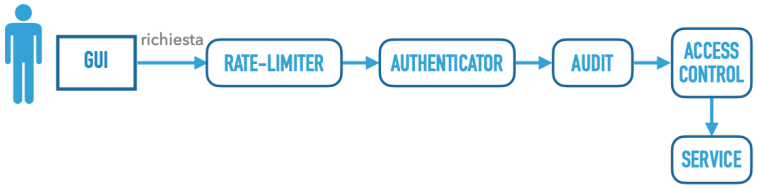


Diagramma UML di sequenza (login):

1. Il Client invia una richiesta al ServerGateway.
2. Il Gateway chiama rateLimiter.acquire() per controllare il traffico.
3. Chiama authenticator.checkCredentials().
4. L'autenticatore verifica l'hash della password (tramite SCryptUtil.check()) usando i dati della UserTable.
5. Se valido, il TokenGenerator crea un token (tokenBuild()).
6. Il token viene salvato nel TokenStore e restituito al client.

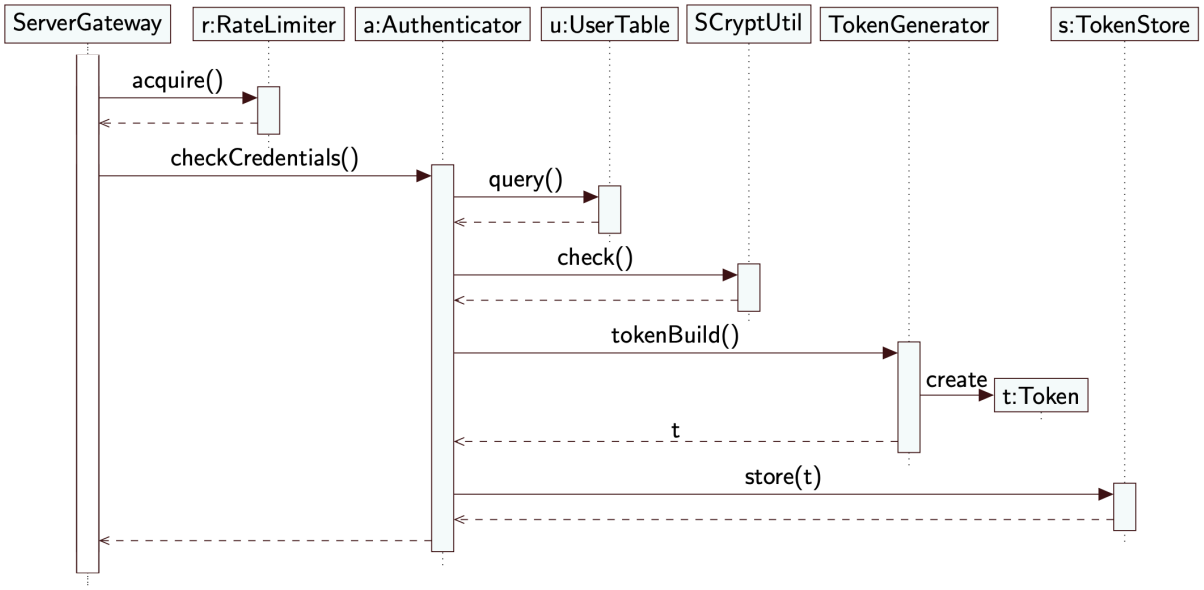


Diagramma UML di sequenza (richieste successive al login)

1. Il gateway controlla sempre prima il limite con acquire().
2. Successivamente, verifica la validità del token inviato dal client interrogando il TokenStore (tramite isValid(token)) prima di procedere.

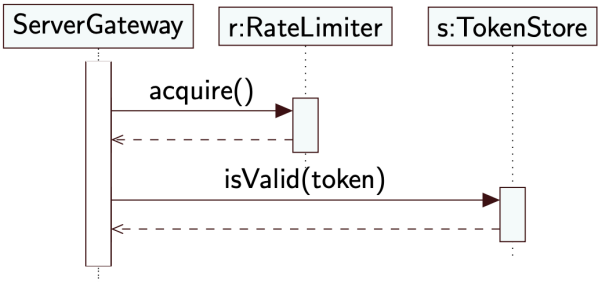
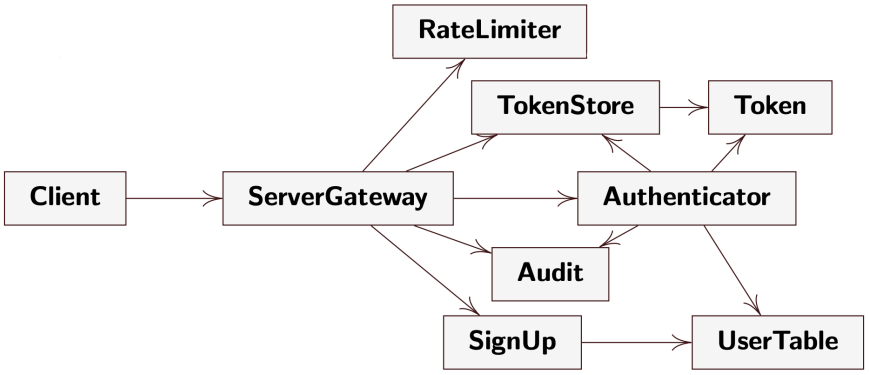


Diagramma UML delle classi



È possibile vedere un esempio del design pattern Authenticator al seguente link: https://github.com/isd-inf/authent/.